STM32F446RE Indoor Air Quality Project

Overview

STM32 Indoor Air Quality Monitoring System is designed to collect indoor environmental measurements, process sensor data, expose the configurations through **Modbus RTU** and store the information in non-volatile memory

The project uses **STM32F446RE** microcontroller and **FreeRTOS** to coordinate application tasks. Incorporates an SGP40 VOC sensor, an SHT3x temperature and humidity sensor, **FRAM** storage and a **Modbus** slave implementation

Features

- ➔ **CMSIS based register-level development:** Direct interaction with STM32 hardware for peripheral configurations and memory mapped registers

- ➔ **Peripheral driver development:** Low level hardware initialization and peripheral components including clock configurations, **GPIO**, **UART** & **DMA**

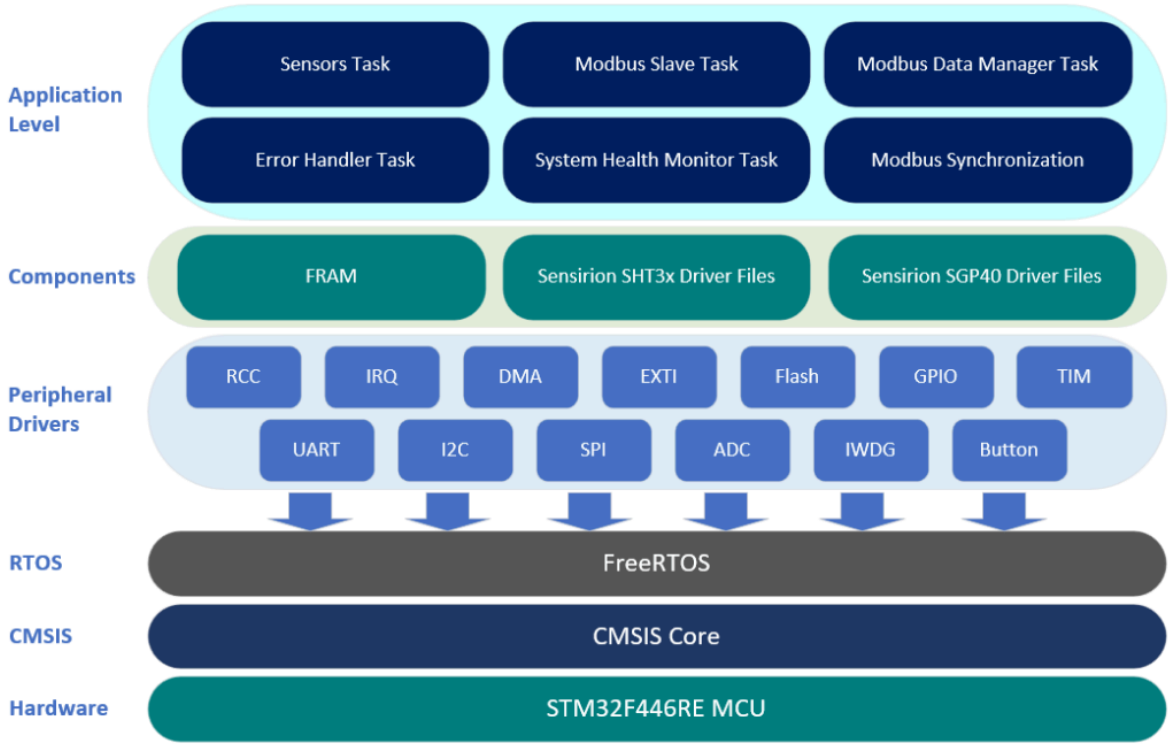
- ➔ **FreeRTOS task design:** Separates major responsibilities into dedicated tasks like **Startup**, **Sensor acquisition**, **Error handling**, **System health monitoring** and **Modbus data management**
    - ◆ FreeRTOS **queues**, **task notifications**, **event groups** and **mutex based synchronization** to coordinate and manage resources between tasks

- ➔ **Environmental Sensor integration**: Measurements are passed to data management layers, that updates the relevant registers and and compares against the configured alarm conditions
- ➔ **Modular Modbus RTU Slave:** Separates request processing from register handling and application data management.
    - ◆ Includes request buffering, CRC validation, response generations and exception responses via **UART**
    - ◆ Write operations include the Data Manager processing, including config updates and persistence

- ➔ **FRAM and Non Volatile Storage:** Fram initialization, validation routines and functions for storing selected configuration and input data, to separate live application state from data needed to survive a power cycle.

- ➔ **Error Handling & System Health Monitoring:** Dedicated components handle firmware error events and health monitoring including **IWDG-related** resets

System Architecture



Base Level

- ➔ **Hardware & CMSIS Core:** Provides a standard interface to the processor and core peripherals and device specific register definitions.
- ➔ **RTOS (FreeRTOS):** Handles Task Scheduling and RTOS synchronization mechanisms

Mid Level

- ➔ **Peripherals Drivers:** UART, GPIO, DMA related functionality that is configured as per requirement.
- ➔ **Components:** External components and vendor specific driver files for external sensors used to integrate the components into the project

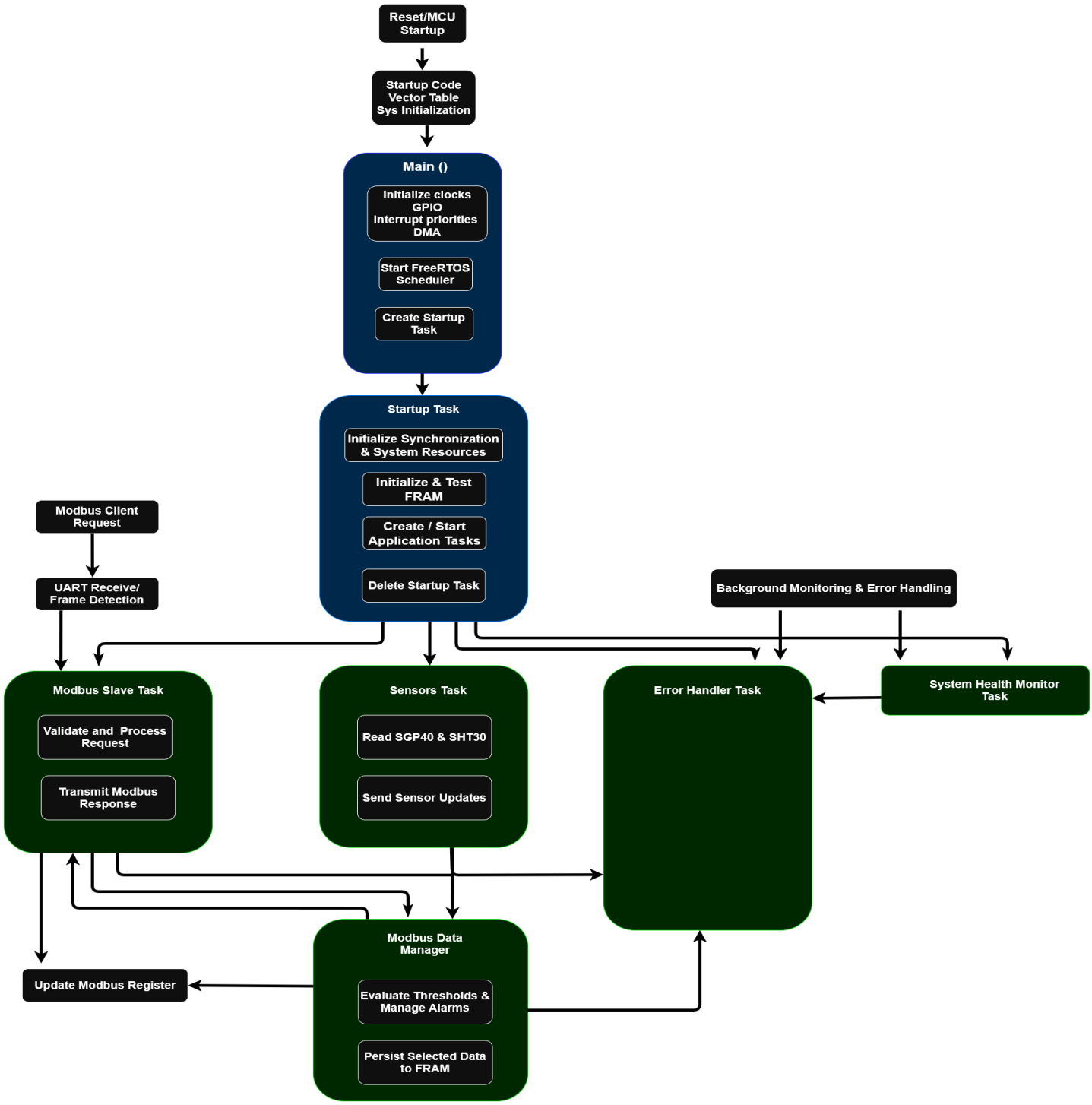
High Level

- ➔ **Application Level:** FreeRTOS tasks for startup, sensor acquisition, Modbus communication, Error Handling and System Health Monitoring.
- ➔ The firmware uses FreeRTOS synchronization according to each component's responsibilities.
  - ◆ Queues transfer structures processing messages between tasks.

- ◆ Task notifications signal events such as received UART data
- ◆ Event Groups coordinate initialization dependencies
- ◆ Mutexes protect relevant shared Modbus data

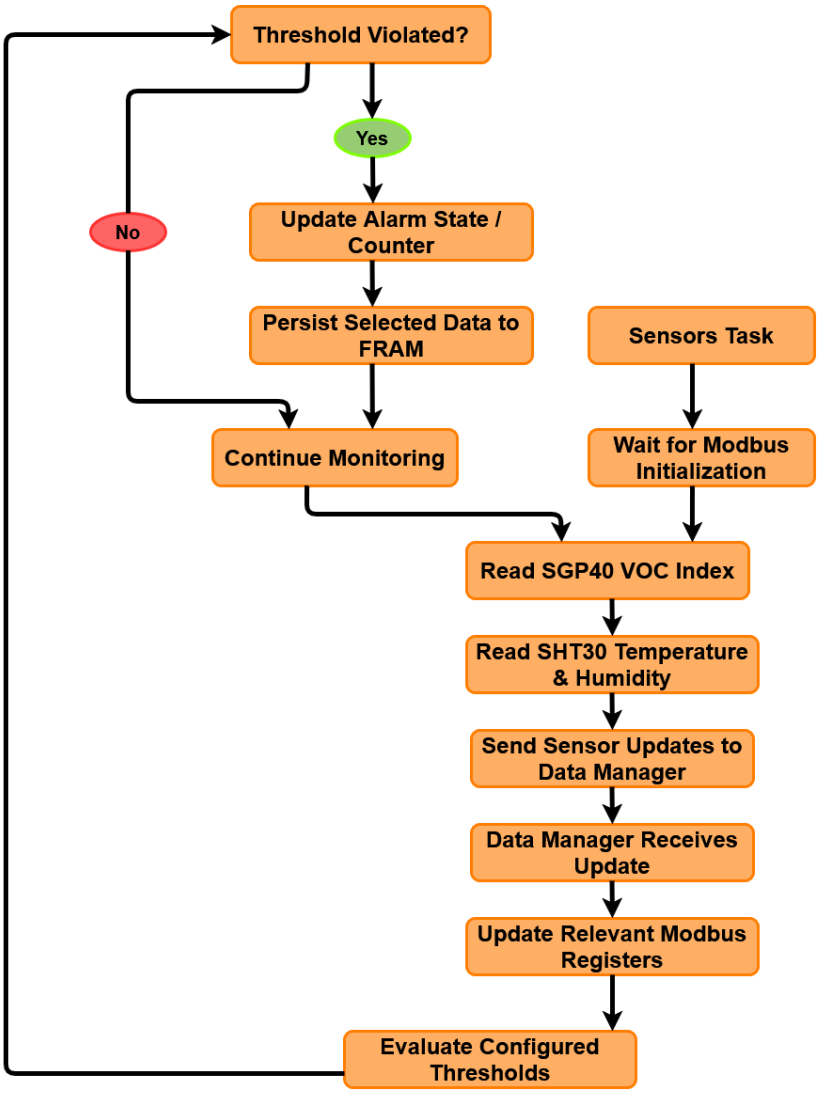
➔ This separation allows peripheral interaction, protocol processing and application level data management to operate independent

Firmware Architecture



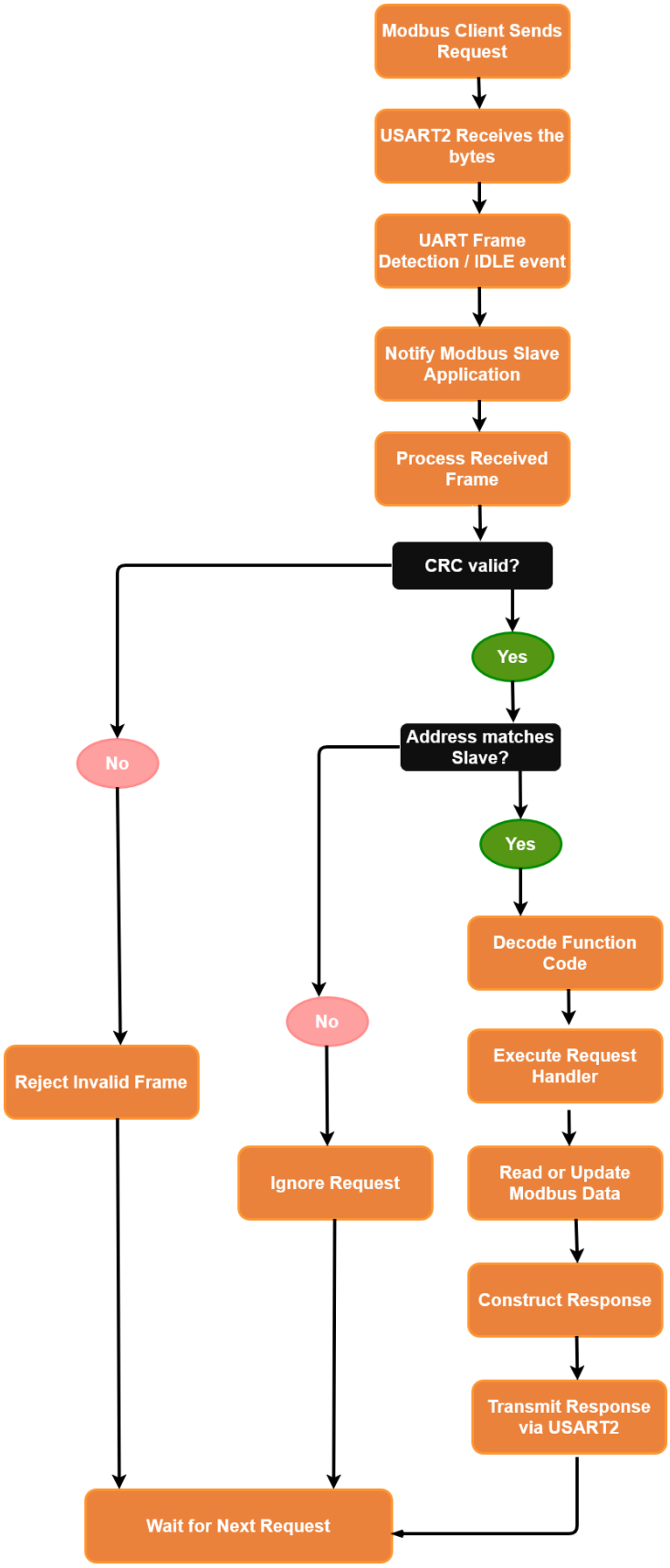
- ➔ **Entry point main()**, performs initial hardware and clock configuration, initializes relevant peripherals and starts the FreeRTOS scheduler
- ➔ **Startup Task** initializes shared resources and starts the application services
- ➔ **Sensors Task** obtains environmental measurements and submits updates to the Modbus Data Manager
- ➔ For **UART** receptions, the interrupt path stores received bytes in a shared receive buffer and a task notification wakes up the Modbus Slave Task when the received frame is ready for processing
- ➔ **Modbus Slave Task** validates incoming frames, checks address and dispatches requests to the handlers. Responses are constructed in a transmit buffer, supplied with a CRC and sent through the UART interface. Write operations that require application level processing are coordinated with the Data Manager.
- ➔ **Modbus Data Manager Task** receives structured messages describing the requested operation, associated data and whether feedback is required. A **Mutex** based synchronization mechanism protects the relevant shared data operations.
- ➔ The Data Manager compares relevant measurements against configured thresholds, and selected configurations and input register data are written to **FRAM** through SPI interface
- ➔ **System Health Monitor** Handles health monitoring by measuring the MCU temperature periodically and also utilises an **Independent Watch Dog** to include reset acknowledgement in case of incorrect behaviour
- ➔ **Error Handler Task** utilizes a FreeRTOS queue that receives error messages and is used for **Error Reporting**. Can be accessed by other application tasks and peripheral drivers making it modular in design.

Sensor Acquisition and Data Management



- ➔ **Sensors Task** waits until Modbus Initialization is complete, reads environmental sensors
- ➔ Sensor updates are passed onto **Modbus Data Manager**
- ➔ The Data Manager updates the shared Modbus data and evaluates configured thresholds
- ➔ When an alarm condition occurs, the relevant alarm state and counter are updated, with selected data persisted to FRAM as applicable.

Modbus RTU request lifecycle



➔ **USART2** receives the incoming serial data

➔ The **UART IDLE** line helps detect the end of the received frame, and the Slave Task is notified

➔ The **Slave task** processes the frame and validates the CRC and slave address

➔ A valid request is dispatched to the function code handler

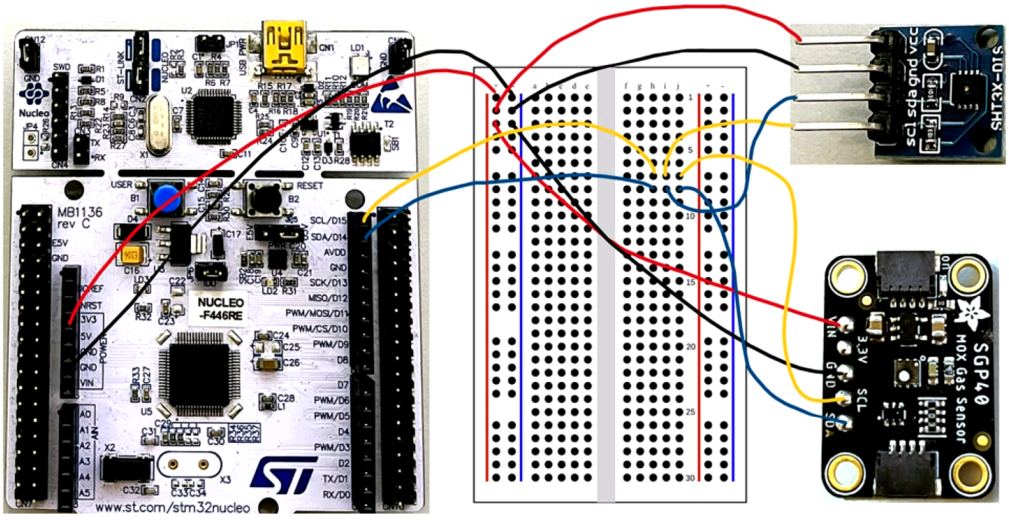
➔ Handler accesses or modifies the relevant Modbus data and constructs the response

➔ Response is transmitted through USART2

Circuit Design

STM32F446RE to sensors

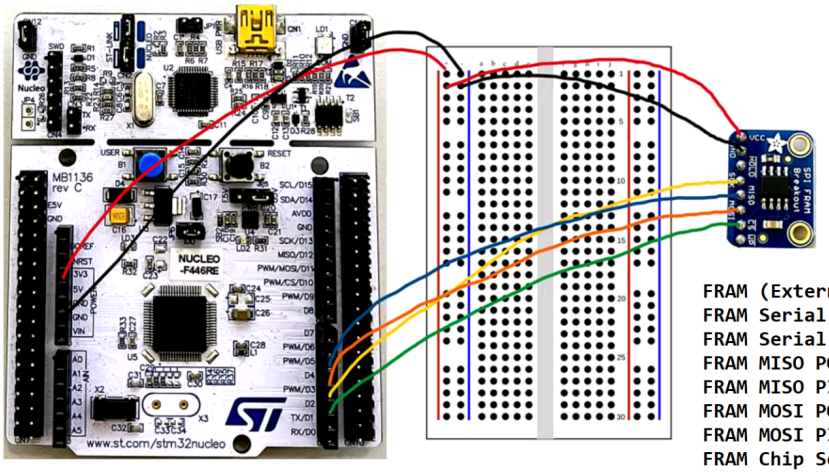
Red: 3.3V, Black: GND, Yellow: I2C SCL, Blue: I2C SDA



Sensirion sensors	
SENSIRION Serial Clock Line PORT	GPIOB
SENSIRION Serial Clock Line PIN	GPIO PIN 8
SENSIRION Serial Data PORT	GPIOB
SENSIRION Serial Data PIN	GPIO PIN 9

STM32F446RE to FRAM

Red: **3.3V**, Black: **GND**, Yellow: **SCK**, Blue: **MISO**, Orange: MOSI, Green: CS

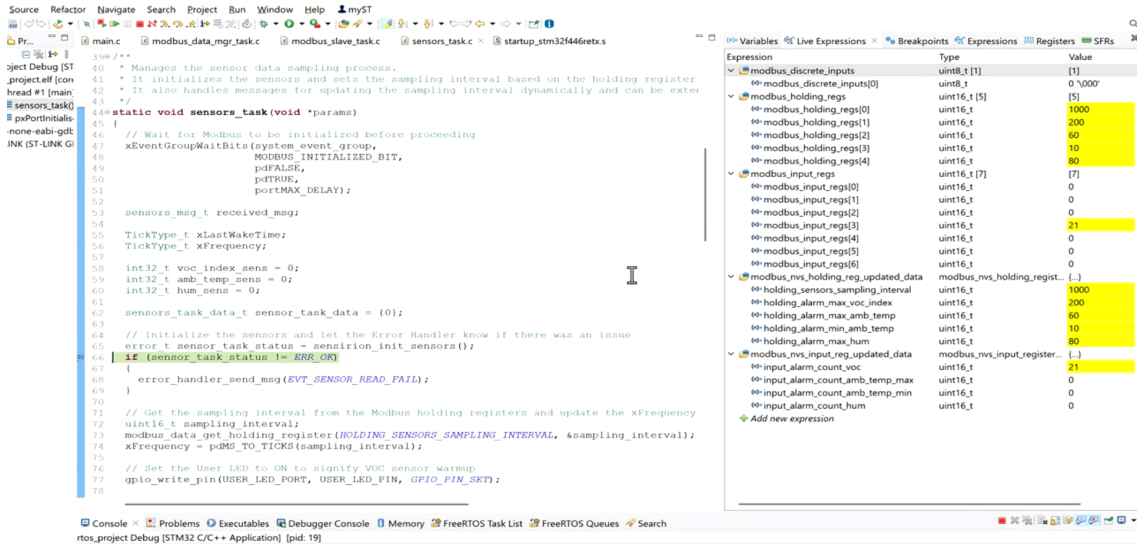


FRAM (External Memory Interface)	
FRAM Serial Clock PORT	GPIOB
FRAM Serial Clock PIN	GPIO PIN 3
FRAM MISO PORT	GPIOB
FRAM MISO PIN	GPIO PIN 4
FRAM MOSI PORT	GPIOB
FRAM MOSI PIN	GPIO PIN 5
FRAM Chip Select PORT	GPIOA
FRAM Chip Select PIN	GPIO PIN 10

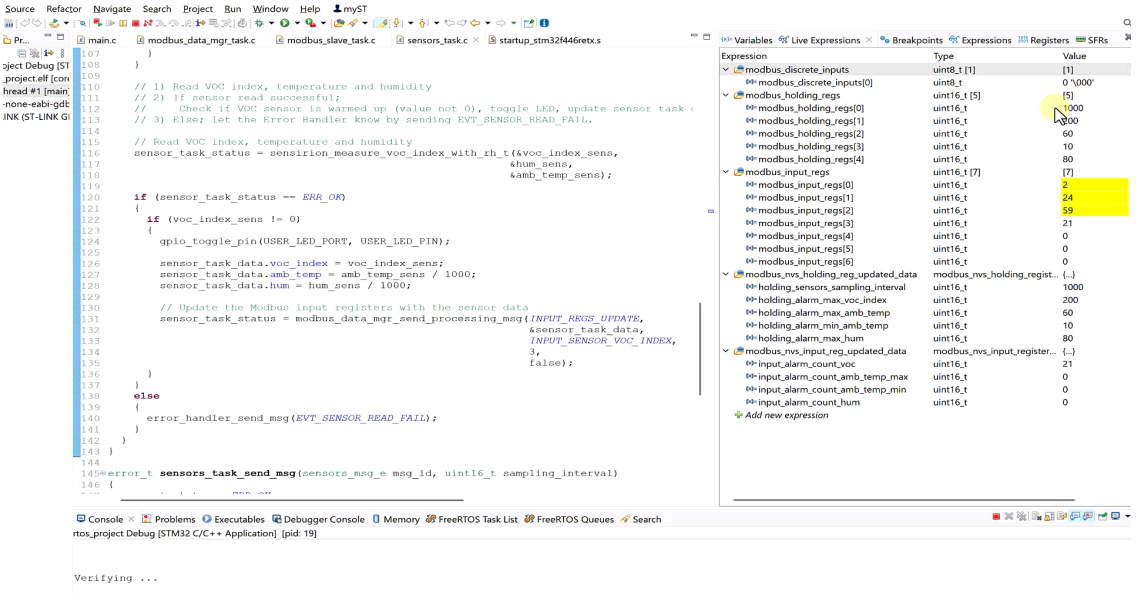
Modbus Testing & Results

➔ Project uses Simply Modbus software application that allows the user PC to operate as a Modbus RTU allowing to connect to Modbus slaves to test and troubleshoot

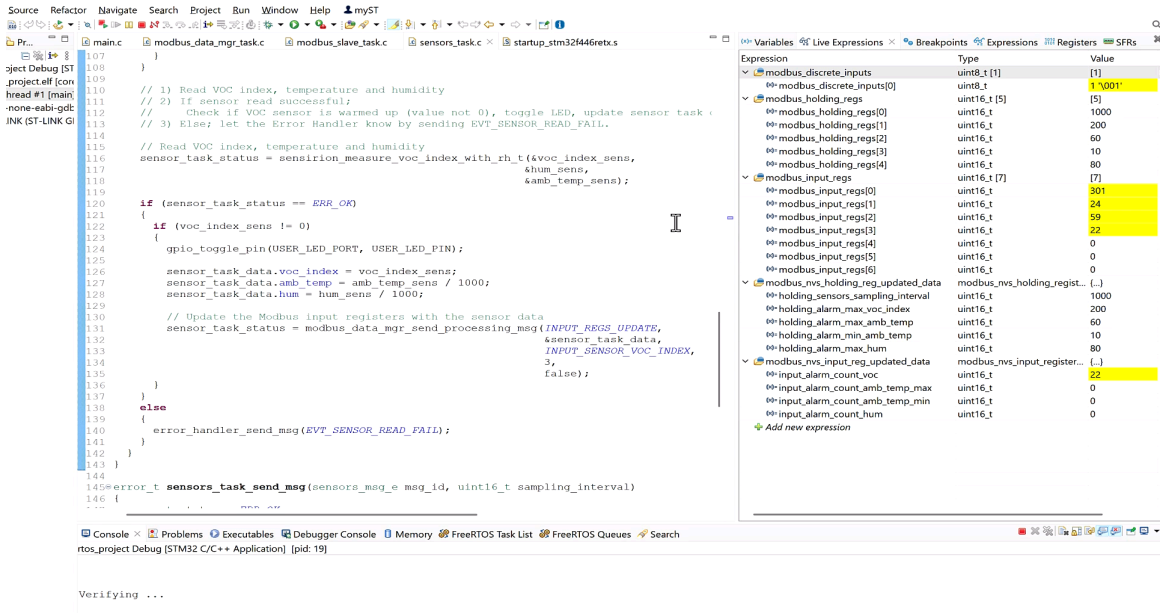
1. Modbus Initialized with holding register values for sampling interval & max



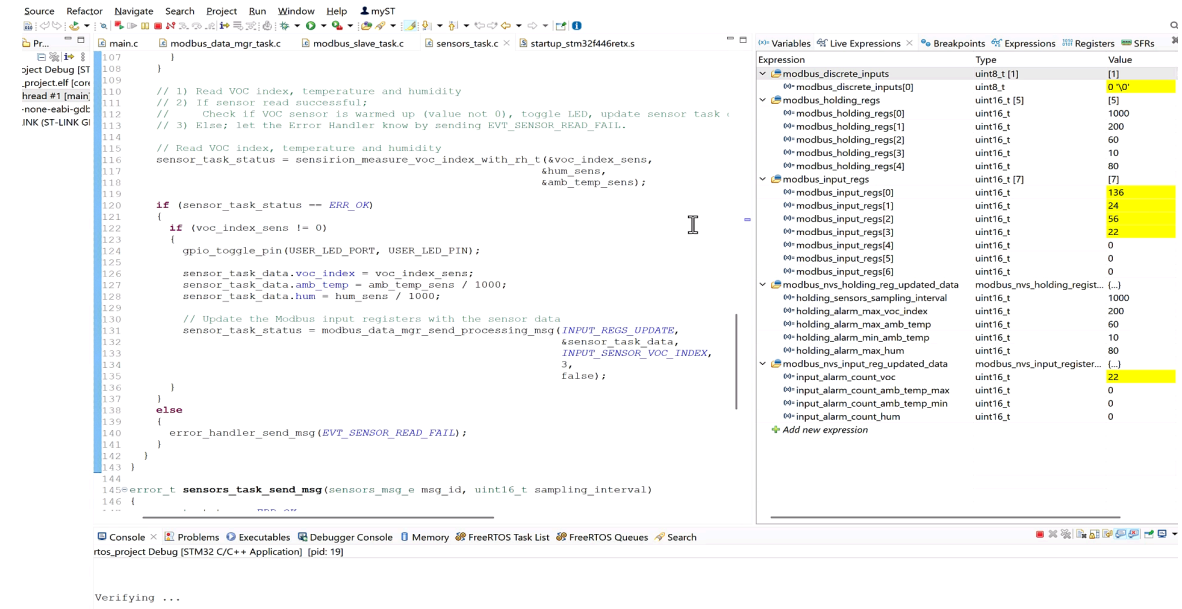
thresholds for voc index, temperature & humidity



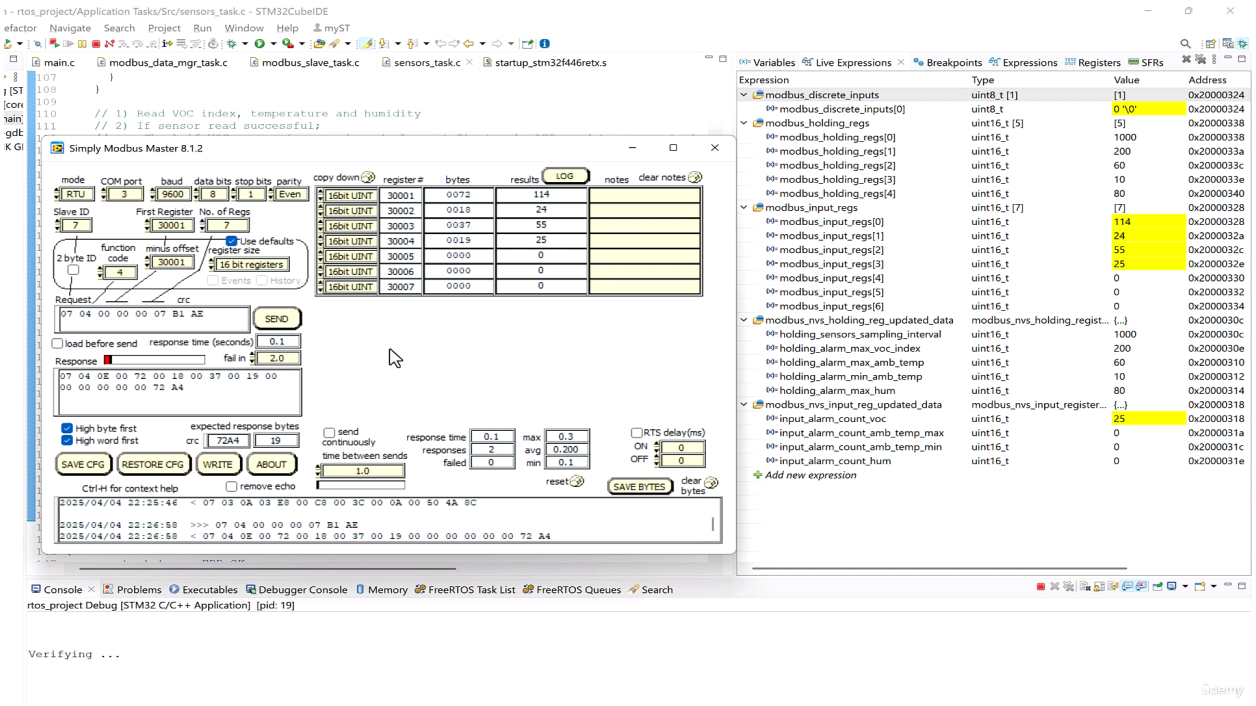
2. Input registers updated with measurements of voc index, temperature and humidity



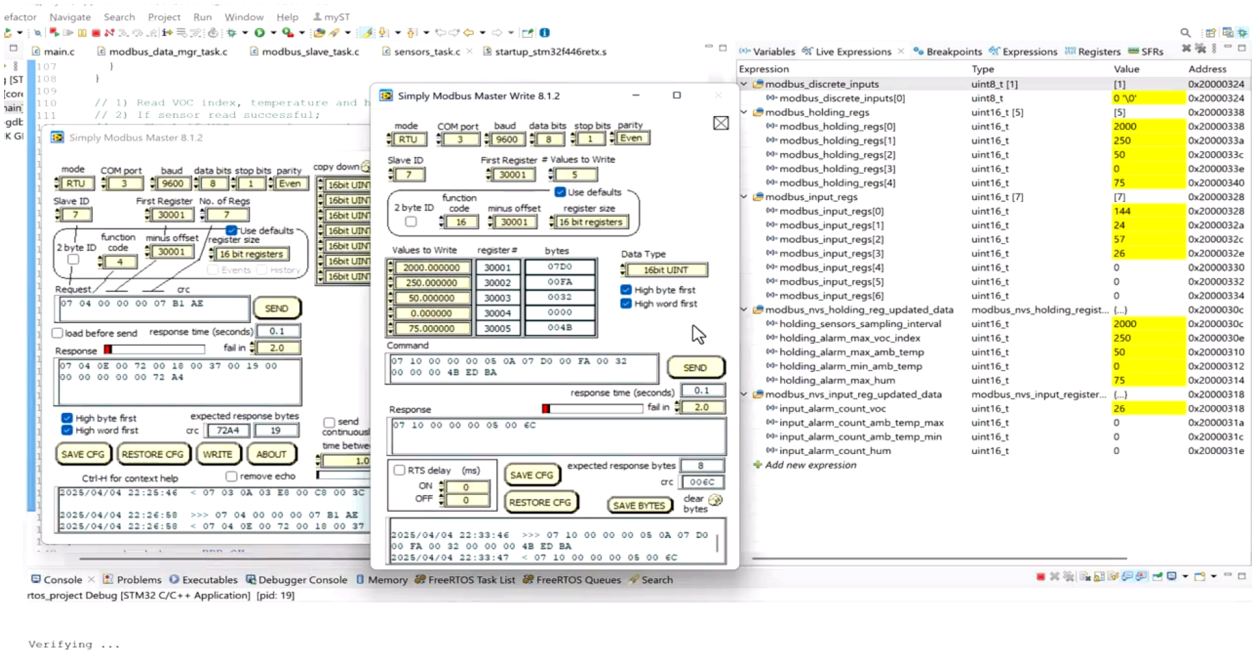
3. Discrete input alarm set when voc index crosses threshold, input register alarm count increases



4. Discrete input alarm reset when voc returns to normal value



5. Simply Modbus Master used to read input registers



6. Updating Holding register values using Simply Modbus Master

Current Limitations

- ➔ **Runtime Diagnostics:** The firmware has dedicated error-handling and health-monitoring tasks, but diagnostic history and error statistics could be exposed more comprehensively to a Modbus client.
- ➔ **Persistence and Data Integrity:** Add a versioned storage layout to support multiple cycles and pull historical data. The current FRAM storage is used for selected configuration and application data. Historical environmental measurements are not documented as a persistent time-series log.
- ➔ **Error & Robustness:** Improve handling of communication timeouts and incomplete frames in Modbus

Future Improvements

- ➔ **System Diagnostics and Fault Recovery:** structured fault-management mechanism that classifies errors by severity and defines appropriate responses. Transient sensor errors could trigger retries. Diagnostic counters and status registers could make these conditions visible to a Modbus client
- ➔ **Remote Diagnostics:** Add Modbus-accessible diagnostic counters and subsystem status information to make sensor faults and communication errors easier to identify from a host system.

Software Stack

- ➔ STM32CUBEIDE development environment based on Eclipse CDT
- ➔ CMSIS, Cortex M core and device register handles
- ➔ FreeRTOS, Task scheduling and synchronization
- ➔ SGP40 and SHT3x Vendor drivers from Sensiron
- ➔ Fujitsu MB85RS64V SPI FRAM module Vendor driver
- ➔ Modbus RTU for Serial client/server communication and register access